Python Assignment
Topic-7: Functions and Methods

Q1. Defining and calling functions in Python.

Ans.

Functions are blocks of code that perform a specific task. They help in dividing the program into smaller parts. In Python, functions are defined using the 'def' keyword. Once defined, a function can be called whenever needed. Functions reduce code repetition and make programs easy to manage. They improve readability and help in debugging. Functions can also return values after execution. Using functions makes the code organized and reusable.

Example:

```
def greet():
   print("Hello")

greet()
```

Q2. Function arguments (positional, keyword, default).

Ans.

Function arguments are values passed to a function when it is called. Positional arguments are passed in the same order as parameters. Keyword arguments are passed using parameter names, so order does not matter. Default arguments have predefined values and are used if no value is passed. Arguments make functions flexible and useful. They allow the same function to work with different data. Understanding arguments is important for writing effective programs.

Example:

```
def add(a, b=5):
   print(a + b)

add(10)
add(10, 2)
```

Q3. Scope of variables in Python.

Ans.

Scope refers to the area where a variable can be accessed. There are mainly two types of scope: local and global. Local variables are defined inside a function and can be used only within that function. Global variables are defined outside functions and can be used anywhere in the program. Proper understanding of scope helps avoid errors. It also helps in writing better and secure code. Managing scope correctly is important in large programs.

Example:

```
x = 10

def show():
    y = 5
    print(x, y)

show()
```

Q4. Built-in methods for strings, lists, etc.

Ans.

Python provides many built-in methods to perform operations on data types like strings and lists. These methods help in modifying, searching, and processing data easily. For example, string methods can change text to uppercase or lowercase. List methods can add or remove elements. Built-in methods save time and reduce the need to write extra code. They are easy to use and very useful in daily programming tasks. Learning these methods helps in writing efficient programs.

Example:

```
name = "python"
print(name.upper())

lst = [1,2,3]
```

```
lst.append(4)
print(lst)
```